

CreditFlow AI Platform

Internship Project — Detailed Requirements Specification

Duration: 7 Days | Team Size: 2 (Backend + Frontend) | Prepared by: Adept Tech Solutions

1. PROJECT OVERVIEW

CreditFlow is a multi-tenant, credit-based SaaS platform for AI-assisted content generation and social publishing. Accounts (individual or team) purchase credits, consume them via AI generation calls, and may buy/sell/transfer credits with other accounts through an internal marketplace. The platform is built as 13 independently deployable backend microservices behind a single API Gateway, plus one frontend application, communicating asynchronously over RabbitMQ and using Redis for caching, JWT session state, task scheduling, and real-time streaming fan-out.

This document specifies functional requirements, data ownership, and required technologies for each service and for the frontend. It does not describe implementation steps — teams are expected to design their own internal architecture within each boundary.

2. SERVICE ARCHITECTURE SUMMARY

#	Service	Primary Data Store	Comm. Style
1	API Gateway	Stateless (Redis-backed)	REST/SSE in, REST/gRPC out
2	Auth Service	PostgreSQL	REST + Redis
3	User / Tenant Service	PostgreSQL	REST + RabbitMQ
4	Billing Service	PostgreSQL	REST + RabbitMQ
5	Credits / Marketplace	PostgreSQL	REST + RabbitMQ
6	Usage / Metering	Redis + PostgreSQL	REST + RabbitMQ
7	AI Generation	PostgreSQL	SSE + RabbitMQ
8	Content Service	PostgreSQL	REST + RabbitMQ
9	Scheduler Service	PostgreSQL + Redis (Celery)	REST + RabbitMQ
10	Social Publishing	PostgreSQL	REST + RabbitMQ
11	Scraper Service	MongoDB	RabbitMQ (job queue)
12	Notification Service	PostgreSQL	RabbitMQ (consumer)
13	Admin / Ops Service	PostgreSQL + Redis reads	REST + RabbitMQ (audit)
—	Frontend Application	N/A (consumes APIs)	REST + SSE (EventSource)

3. PLATFORM-WIDE TECHNOLOGY REQUIREMENTS

Layer	Required Technology
API Gateway / Services	FastAPI (Python) or NestJS (Node.js) — one framework chosen consistently

Relational DB	PostgreSQL (one instance, isolated schema per service)
Document DB	MongoDB (Scraper Service only)
Cache / Session Store	Redis — JWT/jti store, rate limiting, usage counters, SSE pub/sub
Task Queue	Celery + Redis broker (Scheduler Service)
Message Broker	RabbitMQ — topic exchanges per domain (billing_events, social_events, scraper_events, usage_events)
Payments	Stripe (Sandbox / test-mode keys only)
AI Provider (text)	OpenRouter (streaming completions)
AI Provider (image, bonus)	Pollinations.ai or Hugging Face Inference API (free tier, no cost)
Social Integration	LinkedIn API (OAuth 2.0 / OIDC, UGC Posts, Images API)
Scraping	Crawl4AI or Playwright
Email	Resend or Mailgun (free tier, sandbox domain)
Frontend Framework	React (Vite) or Next.js
Frontend Styling	Tailwind CSS
Frontend Calendar UI	FullCalendar or React Big Calendar
Containerization	Docker + docker-compose (local/dev)
CI/CD	GitHub Actions
Deployment Target	AWS free-tier account (EC2 for services/broker/cache, RDS for Postgres) — bonus scope
Version Control Workflow	Git — main (prod), dev (integration), feature/*, fix/* branches

4. FRONTEND APPLICATION REQUIREMENTS

A single frontend application serves three distinct audiences through role-gated routing: the account Owner, Team Members, and the Platform SuperAdmin. Route access is enforced both client-side (hide/redirect) and server-side (Gateway checks role on every call — the frontend restriction is a UX convenience, not a security boundary).

Public / Unauthenticated Pages

- Home / Marketing Page — professional landing page: hero section, feature highlights, pricing tiers, sign-up call to action.
- Sign Up — email + password; on submit, triggers a verification email containing a signed link.
- Log In — email + password; surfaces a 'Forgot password' link.
- Forgot Password (OTP) — request a one-time code by email, enter the OTP, then set a new password. OTP must be short-lived and single-use.
- Email Verification Landing — consumes the emailed verification link/token and activates the account.

Onboarding

- Create or Join Account — new user chooses to create an Individual account, create a Team account, or accept a pending team invite.
- Account Switcher (persistent component) — lets a user move between every account they belong to; triggers a new account-scoped JWT.

Owner-Only Pages

- Owner Dashboard — account-wide summary: usage, credits, team size, plan tier.
- Team Management — invite members, assign/change roles, remove members.
- Billing & Invoices — subscription plan, upgrade/downgrade, invoice history, payment method.
- Credits & Marketplace — buy credits (Stripe checkout), list surplus credits for sale, buy from other accounts.

Owner + Member Pages

- Content Studio — prompt input, streaming AI text output (SSE, token-by-token), optional 'generate image' bonus action, save as draft.
- Calendar / Scheduler — month/week calendar (powered by the Scheduler Service) to place drafted content on a publish date/time, reschedule, or cancel.
- LinkedIn Connections — OAuth connect/disconnect flow, connection status, last-publish status per post.

Platform Admin Console (SuperAdmin Only)

- Cross-account directory — search/browse all accounts on the platform.
- Active sessions viewer — list and revoke active JWTs by account, sourced live from Redis.
- Usage & credits overview — per-account usage and balance, sourced from Usage and Credits services.
- Audit log — searchable timeline of domain events per account, sourced from the Admin Service's audit_log.

Cross-Cutting Frontend Requirements

- JWT handling: store access token in memory, refresh token in an httpOnly cookie; silent refresh on expiry.
- Route guarding by role (Owner / Member / SuperAdmin) with graceful redirect, not just a hidden nav link.
- SSE consumption for AI generation must render tokens as they arrive, not wait for the full response.
- All destructive actions (revoke session, remove member, cancel scheduled post) require a confirmation step.

5. BONUS FEATURES (STRETCH SCOPE)

The following are not required for the core Definition of Done but are recommended stretch goals that meaningfully strengthen the submission.

- AI Image Generation — from the Content Studio, generate an accompanying image for a post using a free image model (Pollinations.ai requires no API key; Hugging Face Inference API free tier is an alternative). Attach the resulting image to the content record.
- Image Publishing to LinkedIn — when a scheduled post has an attached image (AI-generated or manually uploaded), the Social Publishing Service uploads it via LinkedIn's Images API and includes it in the UGC post, producing a text+image post rather than text-only. (important)
- AWS Deployment — deploy the full system to a free-tier AWS account (EC2 for services/broker/cache, RDS for Postgres) via the prod branch pipeline, as described in the Git Workflow section.
- Recurring schedules — allow a scheduled post to repeat on a cadence (e.g. weekly) rather than a single one-off publish time. (important)

6. MULTI-TENANCY & RBAC REQUIREMENTS

All domain data is scoped by account_id, not user_id. An individual signup is an Account of type 'individual' with exactly one member (Owner). A team is an Account of type 'team' with multiple members and roles.

- JWT must carry user_id, account_id, role, and jti.

- Roles: Owner, Admin, Member — enforced at the Gateway on every protected route.
- A user may belong to multiple accounts; the frontend must support an account switcher.
- Credit balance, subscription, and content are shared at the account level; audit attribution (created_by_user_id) is tracked alongside.

7. EVENT RELIABILITY REQUIREMENTS

- Transactional Outbox pattern required in Billing Service: DB state change and 'event to publish' recorded in the same transaction.
- Publisher confirms must be enabled on all RabbitMQ publishes.
- All queues and messages must be durable / persistent (delivery_mode=2).
- Every consumer must implement idempotent processing via a processed_events table keyed on a unique event_id.
- Every queue must have a Dead Letter Exchange configured with a bounded retry count.

8. SERVICE-BY-SERVICE REQUIREMENTS

SERVICE 1 — API GATEWAY

Single entry point for all client traffic. Owns authentication enforcement, request routing to internal services, rate limiting, inbound webhook reception, and SSE re-streaming for AI generation.

Functional Requirements

- Route all frontend/API traffic to the correct downstream service (REST/gRPC).
- Verify JWT on every protected route; reject/refresh as per Auth Service rules.
- Enforce per-account and per-IP rate limiting using Redis token-bucket or sliding-window counters.
- Expose webhook endpoints for Stripe, LinkedIn, and OpenRouter; verify signatures before acting.
- Deduplicate inbound webhook events using a Redis SETNX key on event ID (24h TTL).
- Publish normalized events to RabbitMQ after successful webhook verification and dedup.
- Re-stream Server-Sent Events (SSE) from AI Generation Service to the frontend via Redis pub/sub subscription.
- Aggregate/compose responses where a frontend screen needs data from more than one service.
- Return consistent error schema across all proxied services.

Database Ownership

None — stateless. All state lives in Redis (rate-limit counters, webhook dedup keys, SSE channel subscriptions).

Technology Stack

Layer	Technology
Framework	FastAPI (Python) or Express (Node.js) — pick one, consistent across gateway
Reverse proxy	Nginx or built-in framework routing
Cache/State	Redis
Queue client	RabbitMQ (pika / amqpplib)
Auth	JWT verification middleware (PyJWT / jsonwebtoken)

Event Contract (RabbitMQ)

Publishes: billing.*, social.*, ai.* (relayed from webhooks)

Consumes: none — publisher only

SERVICE 2 — AUTH SERVICE

Owns identity: signup, login, password reset, JWT issuance/refresh, and session revocation state.

Functional Requirements

- Signup with email + password; hash passwords with bcrypt/argon2.
- Email verification flow — generate verification token, emit event for Notification Service to send the email.
- Login endpoint issuing access token (short-lived) + refresh token (long-lived, rotated on use).
- JWT payload must include user_id, account_id, role, and a unique jti.
- Store active jti values in Redis with TTL = token expiry, to support the Admin Service's 'active sessions' view.
- Logout / revoke endpoint that removes the jti from Redis (forces token invalid even before natural expiry).
- Password reset flow via emailed token (short TTL, single-use).

- Rate-limit login attempts per email/IP to mitigate brute force (Redis counter).

Database Ownership

PostgreSQL — tables: users, credentials, refresh_tokens, password_reset_tokens.

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
DB	PostgreSQL
Hashing	bcrypt or argon2
Tokens	JWT (RS256 preferred over HS256 for multi-service verification)
Cache	Redis (jti store, rate-limit counters)

Event Contract (RabbitMQ)

Publishes:user.registered, user.logged_in, user.password_reset_requested

Consumes:none

SERVICE 3 — USER / TENANT SERVICE

Owns accounts (tenants), team membership, invitations, and role assignment. Individual signups and team workspaces are both modeled as an Account.

Functional Requirements

- Create an Account automatically on signup (type: individual); support explicit 'create team' flow (type: team).
- account_members table linking users to accounts with a role: owner / admin / member.
- Team invite flow: generate invite token, emit event for Notification Service to email the invitee.
- Accept-invite endpoint that creates the account_members row and emits member.joined.
- Role update / member removal endpoints, restricted to owner/admin.
- Account switcher support: endpoint returning all accounts a given user belongs to.
- Expose account profile data (name, plan tier, seat count) for the frontend dashboard header.

Database Ownership

PostgreSQL — tables: accounts, account_members, invites.

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
DB	PostgreSQL
Messaging	RabbitMQ consumer + publisher

Event Contract (RabbitMQ)

Publishes:member.joined, account.created, account.updated

Consumes:invoice.paid, user.registered

SERVICE 4 — BILLING SERVICE

Owns all Stripe sandbox interaction: subscriptions, checkout sessions, invoices, proration, and refunds.

Functional Requirements

- Create Stripe Customer on account creation; store stripe_customer_id.
- Create Checkout Session / Subscription for plan selection (Free/Pro/Team tiers).
- Handle plan upgrade/downgrade with Stripe proration.
- Process refund requests; on success, emit event so Credits Service can claw back any associated credit grant.
- Persist every Stripe webhook event received (via Gateway) into an invoices / subscription_events table before further processing — this is the write side of the flow described in section on reliability.
- Implement the Transactional Outbox pattern: DB write and 'event to publish' recorded in the same transaction; a poller publishes to RabbitMQ and marks the row published.
- Dunning handling: on payment_failed, start a grace-period timer; emit subscription.downgraded if unresolved.
- Expose invoice history endpoint for the frontend billing screen (Owner-only).

Database Ownership

PostgreSQL — tables: subscriptions, invoices, outbox_events, refunds.

Technology Stack

Layer	Technology
Payments	Stripe SDK (Sandbox/test mode keys)
Framework	FastAPI or NestJS
DB	PostgreSQL
Messaging	RabbitMQ (topic exchange: billing_events)

Event Contract (RabbitMQ)

Publishes: invoice.paid, payment.failed, subscription.updated, subscription.downgraded, refund.issued

Consumes: none — receives via Gateway-relayed webhook events

SERVICE 5 — CREDITS / MARKETPLACE SERVICE

Owns the credit ledger: purchase, consumption, and peer-to-peer buy/sell/transfer of credits between accounts.

Functional Requirements

- Maintain an append-only ledger table (credits_ledger) — every change is a row, balance is derived, never mutated in place.
- Consume invoice.paid to credit an account's balance according to the plan/pack purchased.
- Marketplace listing: an account can list surplus credits for sale at a chosen price.
- Purchase-from-marketplace endpoint: escrow the buyer's payment intent via Billing Service, transfer credits atomically on confirmation, release funds to seller.
- Idempotency: every consumed event checked against a processed_events table (unique constraint on event_id) before applying — prevents double-crediting on redelivery.
- Expose balance and transaction-history endpoints for the frontend credits screen.

- Emit low-balance and balance-changed events for Usage Service and Notification Service.

Database Ownership

PostgreSQL — tables: credits_ledger, marketplace_listings, processed_events.

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
DB	PostgreSQL (ACID transactions required for transfers)
Messaging	RabbitMQ consumer + publisher

Event Contract (RabbitMQ)

Publishes: credits.credited, credits.debited, credits.low_balance

Consumes: invoice.paid, refund.issued

SERVICE 6 — USAGE / METERING SERVICE

Enforces real-time quota checks and tracks AI usage cost per account, per model.

Functional Requirements

- Redis counter per account for fast quota pre-checks before allowing an AI generation call.
- Consume ai.generation_completed events to record actual token/cost usage into a durable Postgres ledger.
- Reconcile Redis counters against Postgres periodically (or on every write) to prevent drift.
- Emit usage.threshold_reached when an account crosses 80% / 100% of its quota.
- Expose a per-account usage summary endpoint (tokens used, cost, by model) for the Admin dashboard.

Database Ownership

Redis (live counters) + PostgreSQL (usage_ledger, append-only).

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
Cache	Redis
DB	PostgreSQL
Messaging	RabbitMQ consumer + publisher

Event Contract (RabbitMQ)

Publishes: usage.threshold_reached

Consumes: ai.generation_completed

SERVICE 7 — AI GENERATION SERVICE

Wraps OpenRouter for chat/completion calls, streams tokens back via SSE, and stores prompt/response history.

Functional Requirements

- Accept a generation request only after a synchronous quota check against Usage Service (or Redis directly).
- Call OpenRouter's streaming completion endpoint; write each token chunk to a Redis pub/sub channel keyed by job_id.
- Support at least 2 model choices via OpenRouter (e.g. one fast/cheap, one higher quality) selectable by the user.
- Persist full prompt + final response + token counts to Postgres once streaming completes.
- Emit ai.generation_completed (with cost/tokens) and ai.generation_failed (with error reason) on finish.
- Support cancellation of an in-flight stream (job_id-based).
- BONUS: alongside text generation, offer an optional 'generate image for this post' action using a free image model (see Bonus Features section); store the returned image and attach its reference to the content record.

Database Ownership

PostgreSQL — tables: generation_jobs, prompt_history, image_generation_jobs (bonus).

Technology Stack

Layer	Technology
AI Provider	OpenRouter API (streaming)
Framework	FastAPI (async, native SSE/streaming support)
Pub/Sub	Redis pub/sub for token fan-out
Messaging	RabbitMQ publisher
Image Gen (bonus)	Pollinations.ai or Hugging Face Inference API (free tier)

Event Contract (RabbitMQ)

Publishes: ai.generation_completed, ai.generation_failed

Consumes: none

SERVICE 8 — CONTENT SERVICE

Stores and versions generated content (posts), including optional attached images, prior to scheduling and publishing.

Functional Requirements

- Consume ai.generation_completed to create a draft content record when generation was for a 'post' type.
- CRUD endpoints for content: create, edit (versioned), delete, list by account.
- Store an optional image_url / image_asset_ref alongside each content item (from AI-generated image or manual upload).
- Manual image upload endpoint (multipart) for cases where the user supplies their own image instead of generating one.
- Draft/approved/published status machine enforced server-side, not just in the UI.
- Content is scoped by account_id, editable by any member, publishable only by roles with publish permission.
- Emit content.created / content.updated so the Scheduler Service knows what is available to schedule.

Database Ownership

PostgreSQL — tables: content, content_versions.

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
File handling	Multipart upload + object storage (S3 or local volume for dev)
DB	PostgreSQL
Messaging	RabbitMQ consumer + publisher

Event Contract (RabbitMQ)

Publishes:content.created, content.updated

Consumes:ai.generation_completed

SERVICE 9 — SCHEDULER SERVICE

Owns each account's content calendar. Lets users pick a date/time to publish existing content and triggers the publish pipeline when that time arrives.

Functional Requirements

- Calendar endpoint returning all scheduled items for an account within a given date range (month/week view).
- Schedule endpoint: attach a future publish_at datetime to an existing content item (creates a scheduled_posts row — does not duplicate content).
- Reschedule and cancel endpoints for an existing scheduled item.
- Use Celery with Redis as the message broker and result backend for background task execution.
- Celery Beat (or an equivalent periodic task) scans for due scheduled_posts on a fixed interval (e.g. every 60 seconds).
- On a due item, emit content.scheduled so the Social Publishing Service can proceed — the Scheduler Service does not call LinkedIn directly.
- Store schedule times in UTC; convert to the user's local timezone for calendar display on the frontend.
- Prevent double-firing a schedule if the beat task overlaps (use a Redis lock or Celery task idempotency key per scheduled_post id).

Database Ownership

PostgreSQL — table: scheduled_posts. Redis — Celery broker + result backend (separate logical DB index from the caching Redis usage).

Technology Stack

Layer	Technology
Task Queue	Celery
Broker/Backend	Redis
Periodic Trigger	Celery Beat
Framework	FastAPI or NestJS (calendar REST API)
Messaging	RabbitMQ publisher

Event Contract (RabbitMQ)

Publishes:content.scheduled

Consumes:content.created

SERVICE 10 — SOCIAL PUBLISHING SERVICE

Handles LinkedIn OAuth connection, image upload, and publishing of scheduled content to a connected LinkedIn account.

Functional Requirements

- LinkedIn OAuth 2.0 / OpenID Connect connect flow; store encrypted access/refresh tokens per account.
- Each developer registers their own LinkedIn Developer App for local testing — LinkedIn issues API access per app, not shared across the team. Required products: 'Sign In with LinkedIn using OpenID Connect' and 'Share on LinkedIn'. Required scopes: openid, profile, email, w_member_social.
- Consume content.scheduled; if the content item has an attached image, run the image upload flow before creating the post; otherwise create a text-only post.
- Image upload flow: register an upload via LinkedIn's Images API, PUT the binary to the returned upload URL, obtain the resulting asset URN, then reference that URN in the UGC post payload.
- Call LinkedIn's UGC Posts API to publish (text-only or text+image) at the scheduled time.
- Retry with backoff on transient LinkedIn API failures; route to dead-letter queue after max attempts.
- Emit post.published (with LinkedIn post URL/ID) or post.failed (with reason).
- Token refresh job to keep LinkedIn access tokens valid ahead of expiry.

Database Ownership

PostgreSQL — tables: social_connections (encrypted tokens), publish_jobs, post_media.

Technology Stack

Layer	Technology
Integration	LinkedIn API (OAuth 2.0 / OIDC, UGC Posts, Images API)
Framework	FastAPI or NestJS
Encryption	Fernet / libsodium for token-at-rest encryption
Messaging	RabbitMQ consumer + publisher, DLQ configured

Event Contract (RabbitMQ)

Publishes:post.published, post.failed

Consumes:content.scheduled

SERVICE 11 — SCRAPER SERVICE

Runs web-scraping jobs (trend/competitor data) on demand or on schedule, feeding raw data for content generation.

Functional Requirements

- Accept scrape job requests via RabbitMQ (scrape.requested) with a target URL/domain and job type.
- Use a headless browser or crawler (Crawl4AI / Playwright) to fetch and extract structured data.
- Respect robots.txt and reasonable rate limits per target domain.
- Store raw scraped documents in MongoDB (flexible schema for varied page structures).
- Emit scrape.completed with a reference to the stored document(s); emit scrape.failed on error.
- Support scheduled recurring scrape jobs (e.g. daily competitor check) via an internal scheduler.

Database Ownership

MongoDB — collection: scraped_documents.

Technology Stack

Layer	Technology
Scraping	Crawl4AI or Playwright
Framework	FastAPI (job endpoints) + worker process for execution
DB	MongoDB
Messaging	RabbitMQ consumer + publisher

Event Contract (RabbitMQ)

Publishes:scrape.completed, scrape.failed

Consumes:scrape.requested

SERVICE 12 — NOTIFICATION SERVICE

Central place for all outbound email and alerting — signup verification, receipts, invites, failures.

Functional Requirements

- Consume user.registered → send verification email.
- Consume invoice.paid / payment.failed → send receipt / payment-failed email.
- Consume member.joined / account invite events → send invite email.
- Consume post.published / post.failed and usage.threshold_reached → send status/alert emails.
- Use a free-tier transactional email provider (Resend or Mailgun sandbox) — no self-hosted SMTP.
- Optional: Slack incoming webhook for internal ops alerts (no OAuth app review needed).
- Log every notification sent (type, recipient, status) for auditing.

Database Ownership

PostgreSQL — table: notification_log.

Technology Stack

Layer	Technology
Email	Resend or Mailgun (free tier)
Alerts	Slack Incoming Webhooks
Framework	FastAPI or NestJS (mostly a consumer, minimal REST surface)
Messaging	RabbitMQ consumer of all domain events

Event Contract (RabbitMQ)

Publishes:notification.sent

Consumes:user.registered, invoice.paid, payment.failed, member.joined, post.published, post.failed, usage.threshold_reached

SERVICE 13 — ADMIN / OPS SERVICE

Operational visibility layer: active sessions, per-tenant usage and credit dashboards, audit trail, superadmin controls.

Functional Requirements

- List active JWT sessions (jti values) per account by reading Redis directly.
- Revoke a session on demand (delete jti from Redis) — immediately invalidates that token at the Gateway.
- Aggregate per-account view: plan tier, credit balance, usage this period, active members — pulled from Credits, Usage, and User services (read-only calls, no writes).
- Consume all domain events into an audit_log table for a searchable timeline per account (who did what, when).
- SuperAdmin role (platform-level, not account-scoped) can view/search across all accounts; TenantAdmin restricted to their own account_id.
- Expose the data this service surfaces via REST for the frontend admin console.

Database Ownership

PostgreSQL — table: audit_log. Reads Redis and other services' REST APIs directly; owns no other domain data.

Technology Stack

Layer	Technology
Framework	FastAPI or NestJS
DB	PostgreSQL
Cache	Redis (direct read access)
Messaging	RabbitMQ consumer (audit trail) — subscribes to all exchanges

Event Contract (RabbitMQ)

Publishes: none

Consumes: all events, topic-bound (#) for audit purposes

9. GIT WORKFLOW REQUIREMENTS

- Branches: main (production, protected), dev (integration, protected), feature/<name>, fix/<name>.
- All work happens on feature/* or fix/* branches, opened as Pull Requests into dev.
- Every PR requires at least one review from the teammate before merge.
- CI must pass (lint, unit tests, Docker build) before a PR can merge.
- Merges to dev auto-deploy to a shared docker-compose staging environment.
- Merges to main (from dev only, via release PR) trigger the AWS deployment pipeline.
- Commit messages follow Conventional Commits (feat:, fix:, chore:, etc.).

10. DEFINITION OF DONE

- All 13 backend services run independently via docker-compose with their own database/schema, alongside the frontend application.
- Signup → email verification → login → account creation flow works end-to-end.
- Forgot-password OTP flow works end-to-end by email.
- A user can purchase credits via Stripe sandbox checkout and see balance update.

- A user can list credits for sale and another account can purchase them via the marketplace.
- An AI generation request streams tokens live to the frontend via SSE and deducts credits correctly.
- A piece of content can be placed on the calendar via the Scheduler Service and is published to a connected LinkedIn sandbox/test account at the scheduled time.
- At least one scraper job runs and stores data usable by the content flow.
- Owner, Member, and SuperAdmin each see their correct, distinct set of frontend pages, enforced by role.
- Admin console shows active sessions, per-account usage, and an audit log of recent events.
- All inter-service events survive a forced restart of a consumer without data loss or duplication.
- README documents the architecture, setup steps, and any AWS free-tier constraints/tradeoffs made.